Anker Layout 利用ガイド
Mermaid 図版

Mermaid 図で XmForm 風 attachment レイアウトを説明する

Anker Layout Project

Version 0.1.0

2026-06-30

Anker Layout 利用ガイド

Anker Layout は、Qt Widgets で Motif の `XmForm` に近いフォーム型レイアウトを使うための `QLayout` 実装です。各ウィジェットの `left` 、 `right` 、 `top` 、 `bottom` をフォーム、他のウィジェット、位置割合、または自身の初期位置に attachment して配置します。

要件

- Qt 6 Widgets
- C++20
- CMake 3.16 以上

このプロジェクトは C++20 の designated initializer を前提にしています。

```
layout->addWidget(editor, {
    .left = attachWidget(label, 8),
    .right = attachForm(),
    .top = attachOppositeWidget(label),
});
```

ビルド

プロジェクト全体をビルドします。

```
cmake -S . -B build
cmake --build build
```

この環境で Qt License Service のロックエラーが出る場合は、次のように実行します。

```
QTFRAMEWORK_BYPASS_LICENSE_CHECK=1 cmake --build build
```

テストを実行します。

```
QTFRAMEWORK_BYPASS_LICENSE_CHECK=1 ctest --test-dir build --output-on-failure
```

基本的な使い方

`FormLayout` を親ウィジェットに設定し、 `addWidget()` の第 2 引数で `FormConstraints` を渡します。

```
#include "AnkerFormLayout.h"

#include <QApplication>
#include <QLabel>
#include <QLineEdit>
#include <QPushButton>
#include <QWidget>

using namespace Anker;

int main(int argc, char *argv[])
{
    QApplication app(argc, argv);

    QWidget window;
    auto *layout = new FormLayout(&window);
    layout->setContentsMargins(12, 12, 12, 12);

    auto *nameLabel = new QLabel("Name:");
    auto *nameEdit = new QLineEdit;
    auto *okButton = new QPushButton("OK");

    layout->addWidget(nameLabel, {
        .left = attachForm(),
        .top = attachForm(),
    });

    layout->addWidget(nameEdit, {
        .left = attachWidget(nameLabel, 8),
        .right = attachForm(),
        .top = attachOppositeWidget(nameLabel),
    });

    layout->addWidget(okButton, {
        .right = attachForm(),
        .bottom = attachForm(),
    });

    window.resize(420, 160);
    window.show();
    return app.exec();
}
```

制約の考え方

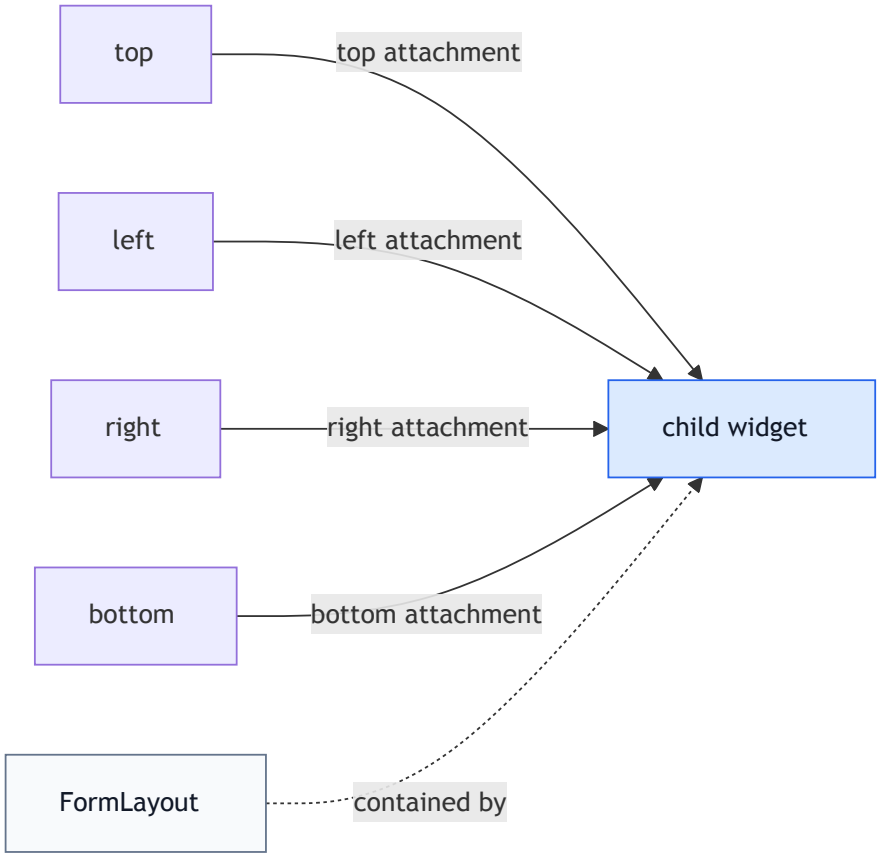
`FormConstraints` は 4 辺の attachment を持つ構造体です。

```
struct FormConstraints
{
    FormAttachment left;
    FormAttachment right;
```

```
    FormAttachment top;
    FormAttachment bottom;
};
```

指定しない辺は `attachNone()` と同じ扱いです。片側だけを指定した場合、ウィジェットは `sizeHint()` と `minimumSize()` を基準に配置されます。両側を指定した場合、その 2 辺の間に伸縮します。

基本モデルは次の図のようになります。



4 辺のうち、指定した辺だけが attachment で解決されます。



Attachment 一覧

attachNone()

辺を固定しません。

```
.left = attachNone()
```

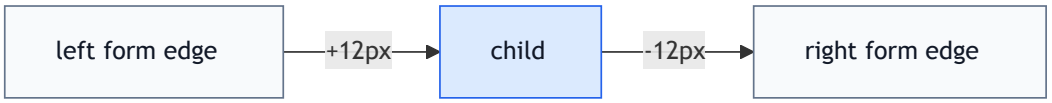
通常は省略すれば同じ意味になります。

attachForm(offset)

フォームの同じ側の辺へ attachment します。

```
layout->addWidget(child, {
    .left = attachForm(12),
    .right = attachForm(12),
    .top = attachForm(12),
});
```

`left` と `top` の offset は内側へ加算されます。`right` と `bottom` の offset は内側へ減算されます。

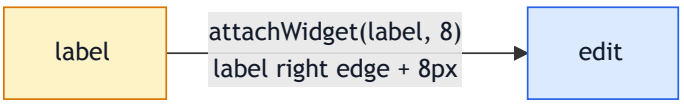


attachWidget(target, offset)

対象ウィジェットの反対側の辺へ attachment します。

```
layout->addWidget(edit, {
    .left = attachWidget(label, 8),
    .top = attachOppositeWidget(label),
});
```

`.left = attachWidget(label, 8)` は、`edit` の左辺を `label` の右辺から 8 px 右へ置きます。



offset を省略した場合は `FormLayout` の spacing を使います。

```
layout->setHorizontalSpacing(12);

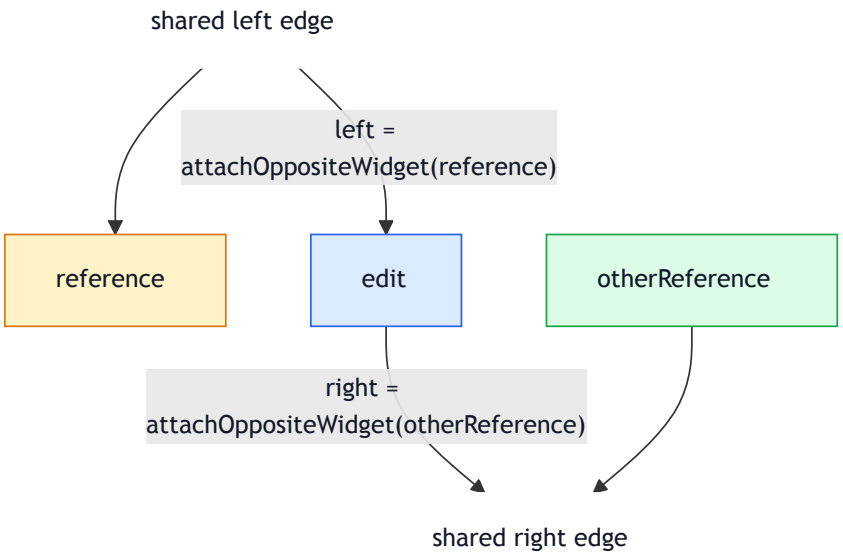
layout->addWidget(rightPanel, {
    .left = attachWidget(leftPanel),
});
```

attachOppositeWidget(target, offset)

対象ウィジェットの同じ側の辺へ attachment します。

```
layout->addWidget(edit, {
    .left = attachOppositeWidget(reference),
    .right = attachOppositeWidget(otherReference),
    .top = attachWidget(reference),
});
```

ラベルと入力欄の上端を揃える、複数行の左端を揃える、といった用途に使います。



attachPosition(position, offset)

フォーム内の割合位置へ attachment します。

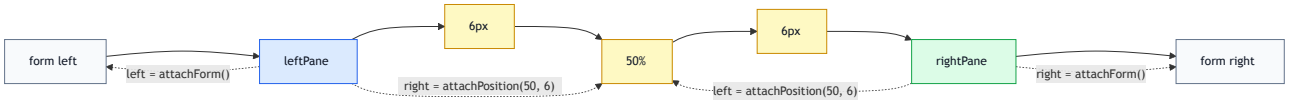
```
layout->setFractionBase(100);

layout->addWidget(leftPane, {
    .left = attachForm(),
    .right = attachPosition(50, 6),
```

```
});

layout->addWidget(rightPane, {
    .left = attachPosition(50, 6),
    .right = attachForm(),
});
```

`fractionBase` が 100 の場合、`attachPosition(50)` はフォーム幅または高さの 50% 位置です。

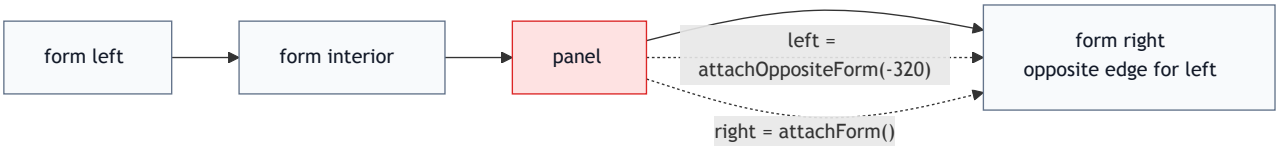


attachOppositeForm(offset)

フォームの反対側の辺へ attachment します。

```
layout->addWidget(panel, {
    .left = attachOppositeForm(-320),
    .right = attachForm(),
});
```

`left` に指定するとフォーム右端側を基準にします。`right` に指定するとフォーム左端側を基準にします。フォーム外側を基準にした配置や、反対側エッジから戻す配置を表現できます。



attachSelf(offset)

レイアウト管理に入る前の自身の初期ジオメトリを基準にします。

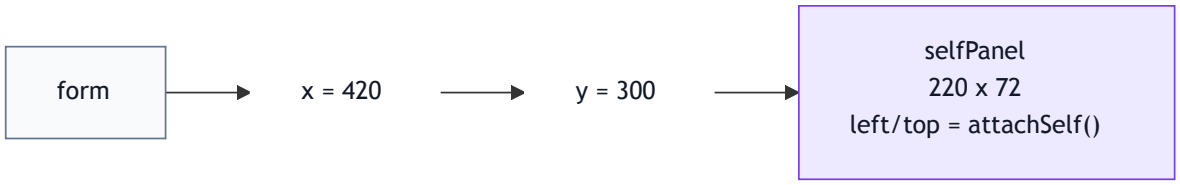
```
selfPanel->setGeometry(420, 300, 220, 72);

layout->addWidget(selfPanel, {
    .left = attachSelf(),
```

```
    .top = attachSelf(),
});
```

`setRubberPositioning(true)` と組み合わせると、フォームサイズ変更時に初期位置の比率を保つように移動します。

```
layout->setRubberPositioning(true);
```



FormLayout の設定

fractionBase

`attachPosition()` の基準値です。デフォルトは `100` です。

```
layout->setFractionBase(100);
```

rubberPositioning

`attachSelf()` の基準位置をフォームサイズに合わせて比率移動させます。デフォルトは `false` です。

```
layout->setRubberPositioning(true);
```

spacing

`attachWidget(target)` の offset を省略した場合に使われます。

```
layout->setHorizontalSpacing(12);
layout->setVerticalSpacing(10);
```

明示的に offset を指定した場合は、その値が優先されます。

```
.left = attachWidget(label, 8)
```

制約の変更

追加済みウィジェットの制約は `setConstraints()` で変更できます。

```
layout->setConstraints(panel, {
    .left = attachForm(12),
    .right = attachForm(12),
    .top = attachPosition(50),
});
```

現在の制約は `constraintsFor()` で取得できます。

```
FormConstraints constraints = layout->constraintsFor(panel);
```

サンプル

各サンプルはルートからまとめてビルドできます。

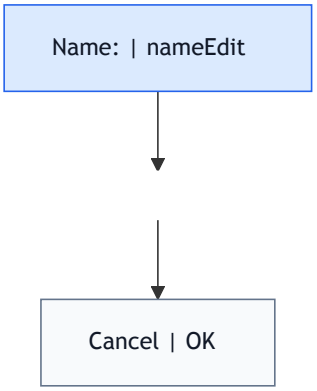
```
cmake -S . -B build
QTFRAMEWORK_BYPASS_LICENSE_CHECK=1 cmake --build build
```

サンプルごとの単体ビルドもできます。

```
cmake -S examples/basic_form -B build-basic_form
QTFRAMEWORK_BYPASS_LICENSE_CHECK=1 cmake --build build-basic_form
```

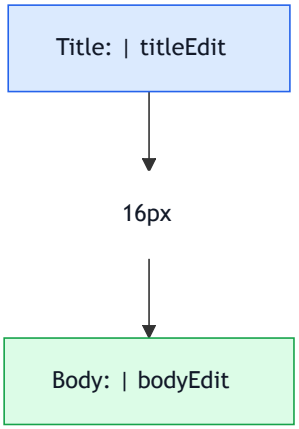
examples/basic_form

ラベル、入力欄、ボタンを使った基本的なフォームです。



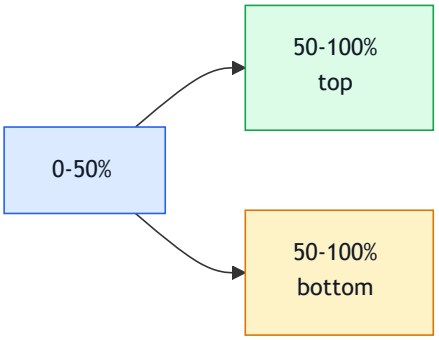
examples/relative_widgets

複数のラベルと入力欄を、他ウィジェットへの attachment で並べます。



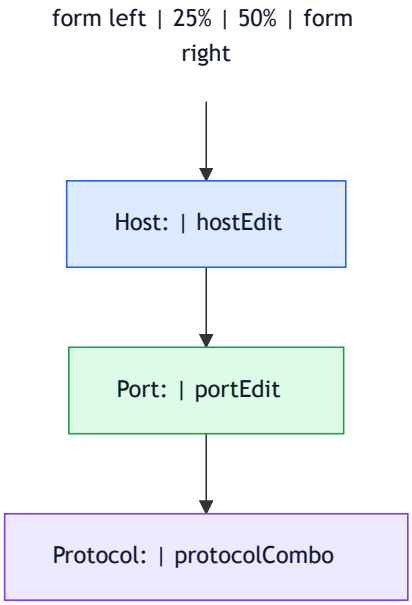
examples/position_anchors

`attachPosition()` と `fractionBase` を使って、フォームを比率で分割します。



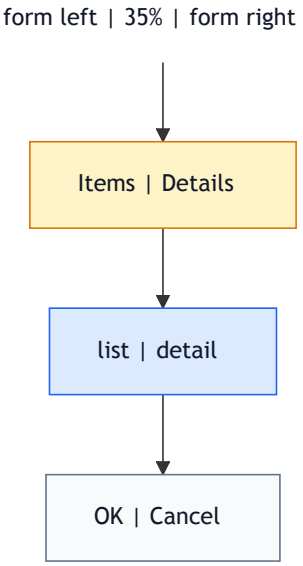
examples/opposite_edges

`attachOppositeWidget()` で複数ウィジェットの辺を揃えます。



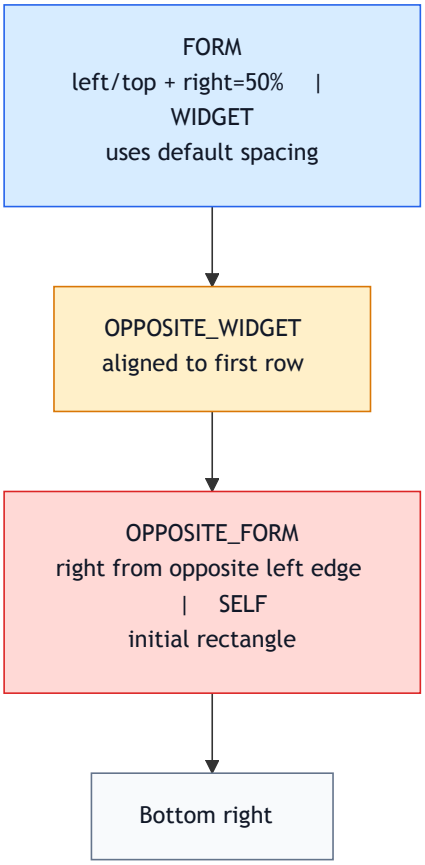
examples/resizable_dialog

リサイズ可能なダイアログで、リスト、詳細欄、ボタンを配置します。



examples/full_attachments

追加機能をまとめて確認するサンプルです。



- `attachOppositeForm()`
- `attachSelf()`
- `setRubberPositioning(true)`
- `setHorizontalSpacing()`
- `setVerticalSpacing()`
- offset 省略時の `attachWidget(target)`

注意点

- 依存関係が循環する attachment は避けてください。レイアウトは複数パスで解決しますが、循環した制約は意図しない配置になる可能性があります。
- `attachSelf()` は、レイアウト管理に入る前の `geometry()` を基準にします。初期位置を指定したい場合は、`addWidget()` の前に `setGeometry()` を呼んでください。
- `attachOppositeForm()` はフォームの反対側エッジを基準にするため、offset の符号によってフォーム外へ配置されます。